

Natural Language Processing, HW4

林彦亨, 111550129

Q1. Method Description (5%)

- Clearly describe your overall tool-calling pipeline and the main idea behind your method.

Ans:

Our final submitted method is a QLoRA fine-tuned Qwen3-14B with hard negative sampling. The main idea is to teach the model to distinguish between semantically similar tools through deliberate exposure to hard negatives during fine-tuning. Rather than relying solely on the model's pre-trained language understanding, as in the prompt-based approach, we fine-tune Qwen3-14B directly on the tool-selection task while constructing training examples that systematically include confusable tool pairs. By ensuring that each training sample contains distractors with high name-token overlap with the correct answer, the model is forced to attend to subtle contextual cues in the task description rather than superficial name similarity.

- Explain your methodology in detail.

Ans:

The training data undergoes three preparation steps before fine-tuning. First, we correct the position bias present in the original data: since correct answers are disproportionately assigned to early labels (A or B), we shuffle the candidate tools and place the correct answer at a uniformly random position, ensuring the answer label is evenly distributed across A–H. Second, because training samples have a variable number of candidate tools while the test set always provides exactly 8, we construct a distractor pool from all unique tools in the training set and pad each sample to 8 options, aligning the training and test formats. Third, we apply hard negative sampling: rather than filling the remaining slots with purely random distractors, 50% are selected by highest name-token overlap with the correct tool, forcing the model to confront difficult, look-alike candidates during training.

The prepared data is used to fine-tune Qwen3-14B via QLoRA, with 4-bit NF4 quantization and LoRA adapters (rank 16) inserted into all attention and feed-forward projection layers, trained on a single RTX 4090 24GB GPU. Two independent adapters are trained separately — one on the full-information configuration and one on the structural-only configuration — so that each adapter is optimized for its respective input format.

At inference time, each test sample is formatted using the same prompt structure as training. The model generates a prediction via greedy decoding with a maximum of 8 new tokens, and the output is parsed with a regex to extract a single uppercase letter (A–H) as the final tool selection.

Q2. Comparison of Methods (10%)

- You are required to implement and evaluate both a Prompt-based approach and a Supervised Fine-Tuning (SFT) approach on the same model . Crucially, you must test BOTH approaches under the following two configurations:
 1. Full-Information Configuration: Provide the complete tool definitions, including tool names, high-level descriptions, and detailed argument schemas.
 2. Structural-Only Configuration: Provide ONLY the tools' parameter keys and data types. You must completely strip out ALL textual descriptions at all levels (including the tool names/descriptions, parameter-level descriptions within arguments, and return-value descriptions within results).
- Within the Prompt-based approach, which configuration performs better? Similarly, within the SFT approach, which configuration performs better? Explain the possible reasons for the differences.

Ans:

Experimental Setup

To ensure a fair comparison, both the prompt-based and SFT approaches are evaluated on the same base model, Qwen3-14B, under two configurations: full-information, which provides complete tool definitions including tool names, descriptions, and detailed argument schemas, and structural-only, which retains only parameter keys and data types with all textual descriptions stripped.

The results

Method	Full-Information	Structural-Only	Gap
Prompt-based	0.90112	0.75988	-0.14124
SFT	0.89830	0.84463	-0.05367

Within Prompt-based: Full-Information Performs Significantly Better.

Under the prompt-based approach, the full-information configuration substantially outperforms the structural-only configuration (0.90112 vs. 0.75988), a gap of 14.1 percentage points.

In a zero-shot setting, the model relies entirely on its pre-trained language understanding to match the task context against candidate tools. With full-information, the model has access to natural language descriptions at every level — the tool name, its purpose, and the semantics of each parameter — making it straightforward to perform context matching between the task description and the tool's stated functionality. Under the structural-only configuration, however, the tool name is hidden and all descriptions are removed, leaving only raw parameter keys and types such as origin: string, destination: string. The model must infer each tool's purpose purely from this structural signature. This is particularly difficult because many tools share similar parameter structures (e.g., multiple tools may have a generic query: string argument), making them nearly indistinguishable without descriptive text. As a result, performance degrades sharply.

Within SFT: Full-Information Still Wins, but the Gap Narrows Substantially.

After fine-tuning, the full-information configuration still outperforms structural-only (0.91525 vs. 0.85875), but the gap shrinks to approximately 5.7 percentage points — less than half of the prompt-based gap.

Fine-tuning on 12,907 training samples allows the model to internalize the mapping between task patterns and tool parameter structures directly from data. Even without any descriptive text, the model learns to associate specific parameter combinations with the types of tasks they serve — for instance, recognizing that a step describing flight search corresponds to a tool with origin, destination, and date parameters. This learned association effectively compensates for the absence of natural language descriptions, explaining why the structural-only SFT model (0.85875) already surpasses the prompt-based full-information model (0.90112) is close and the structural-only prompt-based model by a large margin.

Nevertheless, full-information still holds an advantage. In ambiguous cases where multiple tools share similar parameter structures, the description text provides additional semantic signal that helps the model make the correct final selection. Fine-tuning cannot fully eliminate this reliance on descriptive information, but it significantly reduces it.

Cross-method Observation

Comparing across methods, the SFT structural-only configuration (0.85875) approaches but does not exceed the prompt-based full-information configuration (0.90112), suggesting that fine-tuning can partially compensate for the absence of descriptions but cannot fully substitute for them. More importantly, the best result overall is achieved by combining both: fine-tuning on full-information data with hard negative sampling (0.91525), demonstrating that rich tool semantics and targeted training are complementary rather than redundant.

Q3. Overcoming Tool Ambiguity and Misselection (10%)

- In real-world tool calling, many candidate APIs share highly similar names or overlapping descriptions (e.g., `train_ticket_query` vs. `search_train`), leading to frequent misselection errors by the LLM.
- Describe the specific strategies you implemented to resolve this ambiguity and improve the model's ability to distinguish between highly similar tools
- Quantitative Error Statistics: You must provide a statistical breakdown of your model's failure cases. Specifically, calculate the total count and percentage of errors that were caused by this tool ambiguity issue (e.g., provide the proportion of tool-misselection errors relative to your total incorrect predictions).

Ans:

Quantitative Error Statistics

To quantify tool ambiguity errors, we ran the prompt-based model (Qwen3:8B, full-information configuration) on a held-out evaluation split of 680 samples drawn from the training data. Each error was classified as an ambiguity error if the predicted tool shared high name-token overlap with the correct tool, measured by either string similarity (SequenceMatcher ratio ≥ 0.5) or Jaccard overlap of underscore-split token sets (≥ 0.5).

Out of 40 total errors, 29 were caused by tool ambiguity (72.5% of all errors, 4.26% of total samples). The remaining 11 errors were caused by other factors such as context misinterpretation or out-of-distribution tool combinations.

Category	Count	% of Errors	% of Total
Ambiguity errors	29	72.5%	4.26%
Other errors	11	27.5%	1.62%
Total errors	40	100%	5.88%

The most frequently confused tool pairs are shown below:

Correct Tool	Predicted Tool	Occurrences
file_modify / file_write	file_write / file_modify	7
hotel_booking_query	hotel_booking, hotel_booking_modification	3
flight_search	flight_ticket_changing, search_flights	3
appliance_repair_*	other appliance_repair_* variants	3
car_rental_query	car_rental_changing, car_rental_cancelling	2

These results confirm that tool misselection is the dominant failure mode, and that the most problematic cases involve tools within the same domain that share a common name prefix (e.g., car_rental_*, appliance_repair_*) or near-synonymous names (e.g., file_modify vs. file_write).

Strategies to Resolve Tool Ambiguity

Strategy 1: Hard Negative Sampling

The primary strategy is to directly expose the model to confusable tool pairs during fine-tuning. In our standard SFT data preparation, distractor tools are sampled randomly from a pool of all unique tools in the training set. While this ensures format consistency, it rarely forces the model to distinguish between similar-named tools within the same sample.

In the hard negative variant (sft_hard), 50% of the distractor slots are filled by selecting tools with the highest name-token overlap with the correct tool, scored by Jaccard similarity on underscore-split tokens:

```
def name_overlap(a, b):  
    pa = set(a['name'].lower().split('_'))  
    pb = set(b['name'].lower().split('_'))  
    return len(pa & pb) / max(len(pa | pb), 1)
```

The remaining 50% are still randomly sampled to maintain diversity. This forces the model, during every training step, to read the task context carefully and distinguish the correct tool from its closest look-alike competitors, rather than selecting based on superficial name matching alone.

The effect is directly visible in the results: SFT with hard negatives improves over vanilla SFT by +1.695 pp on the full-information configuration (0.91525 vs. 0.89830) and +1.412 pp on structural-only (0.85875 vs. 0.84463).

Strategy 2: Position-Unbiased Training

A secondary source of error is positional bias: if the model learns that the correct answer tends to appear early in the option list, it may select tool A or B regardless of semantics. This would mask ambiguity errors by making the model appear to "guess correctly" for the wrong reason, while also artificially inflating errors when the correct answer happens to be in a later position.

By shuffling all options and placing the correct answer at a uniformly random position before fine-tuning, we ensure the model cannot exploit positional shortcuts and must resolve tool selection based on semantic understanding alone.